

Projet : Mini CRM (M-Atici)

Livrable : B2-CP4 - Accès aux données SQL et NoSQL

Date : 23 Avril 2026

Documentation Technique : Accès aux données sécurisé

1. Architecture globale

Pour mon CRM (M-Atici), j'utilise deux types de bases de données : PostgreSQL pour le stockage principal (relationnel/SQL), et Redis en tant que base NoSQL pour mon système de cache. Le tout tourne dans des conteneurs isolés via Docker.

2. ORM et requêtes préparées

Pour communiquer avec PostgreSQL, j'ai choisi l'ORM Prisma. Ce choix garantit directement la sécurité des requêtes :

- Les requêtes générées par Prisma sont nativement préparées, ce qui bloque toutes les attaques par injection SQL. Je n'ai pas besoin d'échapper les variables à la main.
- Le typage strict du schéma permet de s'assurer qu'on insère vraiment ce qui est attendu en base.

3. Implémentation du CRUD complet

Toutes les actions CRUD classiques (créer, lire, modifier, supprimer) sont fonctionnelles sur les entités principales de l'application (Clients et Devis). Par exemple, l'insertion sécurisée d'un client se fait avec cette méthode dans mon contrôleur :

```
await prisma.client.create({ data: { nom, prenom, email, telephone, entreprise, adresse, userId } })
```

4. Introduction NoSQL (Redis)

Pour valider l'utilisation du NoSQL, j'ai ajouté Redis à mon infrastructure. Mon cas d'usage est un système de cache pour l'API.

Quand un utilisateur demande sa liste de clients (`GET /api/clients`), le serveur cherche d'abord dans la RAM de Redis. Si la donnée y est, ça va beaucoup plus vite et on évite de solliciter PostgreSQL. Dès qu'un client est modifié ou ajouté, j'invalides ce cache avec la commande Redis `del` pour que la prochaine requête récupère bien les nouvelles données en base.

5. Sécurisation des accès aux données

Au-delà des injections SQL, j'ai renforcé l'accès aux données pour respecter le principe de moindre privilège :

- Il faut avoir un token JWT valide pour accéder à l'API.
- Surtout, pour chaque requête de modification ou de lecture en base de données, je filtre systématiquement avec l'ID de l'utilisateur connecté (`userId: req.utilisateurId`). C'est comme ça que je m'assure qu'aucun compte ne peut voir ou manipuler les clients/devis d'un autre compte.

6. API et Swagger

Enfin, j'ai utilisé `swagger-ui-express` et `swagger-jsdoc` pour générer automatiquement une page de documentation OpenAPI (accessible sur `/api-docs`).

Ça me permet de lister toutes les routes disponibles de mon backend, de voir les paramètres attendus, et de tester mes requêtes CRUD en direct sans avoir besoin de mon interface React.